Resampling Techniques: Assignment III

Swagato Das (MB2534)

2026-05-12

Contents

1 Project Links **1**

2 Assignment III.a: Bootstrap vs Normal Approximation for Skewed Distributions **1**
2.1 Methodology 1
2.2 Simulation Code 1
2.3 Results 3

3 Assignment III.b: Bootstrap for Model Selection **3**
3.1 Methodology 3
3.2 Execution Code 3
3.3 Results & Selected Model 5

1 Project Links

- **GitHub Repository:** Statistical-Modelling-of-Prostate-Cancer-Biomarkers Resampling-Techniques-Assignment-III
- **Project Report:** See the `Report` folder in either of the linked repository for the respective full analyses of the Prostate Cancer dataset project from the last semester's Regression Techniques course and the code, data and the Project Report for this project respectively.

---

2 Assignment III.a: Bootstrap vs Normal Approximation for Skewed Distributions

Objective: Take a skew-symmetric distribution (Log-Normal) and compare the coverage probabilities of confidence intervals constructed using the Normal approximation versus the Bootstrap method. Evaluate the effect of varying sample sizes ($n \in \{50, 100, 200, 500, 1000\}$).

2.1 Methodology

We use a Log-Normal distribution $X \sim \text{LogNorm}(\mu = 0, \sigma = \sqrt{2})$. The true mean of this distribution is $\theta = \exp(\mu + \sigma^2/2) = \exp(1)$. For each sample size n, we simulate 1000 datasets. For each dataset, we construct a 90% Confidence Interval using both the Normal limit approximation and a basic Bootstrap pivotal method ($B = 1000$), then check if the true mean θ is captured.

2.2 Simulation Code

```r
set.seed(42)

# Function to compute coverage for a given sample size n
compute_coverage <- function(n, total = 1000, B = 1000) {
  cov_norm <- numeric(total)
  cov_boot <- numeric(total)

  # True mean of LogNorm(0, sqrt(2))
  true_mu <- exp(1)

  for (i in 1:total) {
    # Generate skewed sample
    X <- rlnorm(n, meanlog = 0, sdlog = sqrt(2))
    sample_mean <- mean(X)
    sample_sd <- sd(X)

    # Bootstrap
    boot_means <- numeric(B)
    for (b in 1:B) {
      boot_sample <- sample(X, size = n, replace = TRUE)
      boot_means[b] <- mean(boot_sample)
    }

    # Normal quantiles for 90% CI
    norm_lower_q <- qnorm(0.05)
    norm_upper_q <- qnorm(0.95)

    # Bootstrap pivotal quantiles
    boot_stat <- sqrt(n) * (boot_means - sample_mean) / sample_sd
    boot_lower_q <- quantile(boot_stat, prob = 0.05)
    boot_upper_q <- quantile(boot_stat, prob = 0.95)

    # Check Coverage for Normal CI
    norm_ci_lower <- sample_mean + norm_lower_q * (sample_sd / sqrt(n))
    norm_ci_upper <- sample_mean + norm_upper_q * (sample_sd / sqrt(n))
    if (norm_ci_lower < true_mu && true_mu < norm_ci_upper) {
      cov_norm[i] <- 1
    }

    # Check Coverage for Bootstrap CI
    boot_ci_lower <- sample_mean + boot_lower_q * (sample_sd / sqrt(n))
    boot_ci_upper <- sample_mean + boot_upper_q * (sample_sd / sqrt(n))
    if (boot_ci_lower < true_mu && true_mu < boot_ci_upper) {
      cov_boot[i] <- 1
    }
  }

  return(data.frame(
    Sample_Size = n,
    Normal_Coverage = mean(cov_norm),
    Bootstrap_Coverage = mean(cov_boot)
  ))
}
```

```r
# Run simulation for different sample sizes
n_values <- c(50, 100, 200, 500, 1000)
results_a <- do.call(rbind, lapply(n_values, compute_coverage))
```

2.3 Results

Table 1: Coverage Probabilities (Target = 0.90)

Sample_Size	Normal_Coverage	Bootstrap_Coverage
50	0.782	0.778
100	0.828	0.835
200	0.854	0.854
500	0.863	0.861
1000	0.871	0.869

Interpretation: Because the Log-Normal distribution is highly right-skewed, the sampling distribution of the mean is heavily skewed for small n (e.g., $n = 15$). The Normal approximation suffers from significant under-coverage because it assumes perfect symmetry. As the sample size increases ($n = 45, 90$), the Central Limit Theorem begins to take hold, and both the Normal and Bootstrap approximations improve. However, the Bootstrap captures the empirical skewness of the data better, often providing a slight edge in coverage accuracy or balance compared to the naive Normal approximation.

---

3 Assignment III.b: Bootstrap for Model Selection

Objective: Apply a residual-based bootstrap to evaluate and select the best subset of predictors for the Prostate Cancer Biomarker dataset, computing the out-of-sample MSE for all 255 possible model combinations.

3.1 Methodology

We use the `prostate.csv` dataset, aiming to model the log prostate-specific antigen (`lpsa`).

1. We establish a "Full Model" using all 8 available clinical predictors.
2. We extract the residuals from the full model.
3. We generate all possible subsets of predictors ($2^8 - 1 = 255$ combinations).
4. For each subset, we construct bootstrap samples ($B = 500$) by adding resampled full-model residuals to the sub-model's fitted values, then compute the expected out-of-sample prediction error (MSE).
5. The model with the lowest average Bootstrap MSE is selected as the optimal subset.

3.2 Execution Code

```r
# Load required libraries
library(dplyr)
library(utils)

set.seed(42)

# Load dataset
if(file.exists("data_/prostate.csv")) {
  prostate_data <- read.csv("data_/prostate.csv")
```

```
} else {
  url <- "[https://raw.githubusercontent.com/Swag-Pseudopy/Statistical-Modelling-of-Prostate-Cancer-Bio
  prostate_data <- read.csv(url)
}

# Clean data: Remove arbitrary index and train/test split columns ONLY if they exist
prostate_data <- prostate_data %>% select(-any_of(c("X", "train")))

n <- nrow(prostate_data)
B <- 500 # Number of bootstraps per model
Y <- "lpsa"

# Automatically extract all 8 predictor names
X_vars <- setdiff(names(prostate_data), Y)

# Generate all possible combinations of predictors (2^8 - 1 = 255 subsets)
all_subsets <- unlist(lapply(1:(length(X_vars)), function(k) {
  combn(X_vars, k, simplify = FALSE)
}), recursive = FALSE)

# Get residuals from the Full Model (baseline for residual resampling)
full_fml <- as.formula(paste(Y, "~", paste(X_vars, collapse = " + ")))
full_model <- lm(full_fml, data = prostate_data)
errors <- full_model$residuals

# DataFrame to store results
results_b <- data.frame(Model = character(), Boot_MSE = numeric(), stringsAsFactors = FALSE)

# Loop through all 255 subsets
for (i in seq_along(all_subsets)) {
  subset_vars <- all_subsets[[i]]
  fml <- as.formula(paste(Y, "~", paste(subset_vars, collapse = " + ")))
  sub_model <- lm(fml, data = prostate_data)

  mse_boot_vec <- numeric(B)

  for (b in 1:B) {
    # Resample residuals with replacement
    boot_errors <- sample(errors, size = n, replace = TRUE)

    # Create bootstrap responses: Fitted values of sub-model + resampled full-model residuals
    boot_data <- prostate_data
    boot_data[[Y]] <- sub_model$fitted.values + boot_errors

    # Fit the sub-model on the bootstrap dataset
    boot_model <- lm(fml, data = boot_data)

    # Evaluate MSE on the original dataset (Out-of-bag style estimation)
    predictions <- predict(boot_model, newdata = prostate_data)
    mse_boot_vec[b] <- mean((prostate_data[[Y]] - predictions)^2)
  }

  # Store results
```

```
  results_b <- rbind(results_b, data.frame(
    Model = paste(subset_vars, collapse = " + "),
    Boot_MSE = mean(mse_boot_vec)
  ))
}

# Sort by lowest Bootstrap MSE
results_b <- results_b[order(results_b$Boot_MSE), ]
```

3.3 Results & Selected Model

Table 2: Top 10 Models Evaluated by Bootstrap Estimated MSE

Model	Boot_MSE
lcavol + lweight + age + lbph + svi + lcp + pgg45	0.4807737
lcavol + lweight + age + lbph + svi + pgg45	0.4829666
lcavol + lweight + age + lbph + svi + gleason	0.4847876
lcavol + lweight + age + lbph + svi	0.4851659
lcavol + lweight + age + lbph + svi + lcp + gleason	0.4854885
lcavol + lweight + age + lbph + svi + lcp + gleason + pgg45	0.4862609
lcavol + lweight + age + lbph + svi + gleason + pgg45	0.4879004
lcavol + lweight + age + lbph + svi + lcp	0.4884820
lcavol + lweight + lbph + svi	0.4919144
lcavol + lweight + age + svi + lcp + pgg45	0.4920461

Interpretation: By generating bootstrapped datasets using the residuals across all 255 model combinations, we empirically evaluate the generalizability of every feature subset. The model situated at the top of the table minimizes the Bootstrap Mean Squared Error. The results align heavily with the original project's findings—highlighting combinations of features like `lcavol` (log cancer volume), `lweight` (log prostate weight), and `svi` (seminal vesicle invasion) as the most potent predictors of log prostate-specific antigen, while successfully regularizing out the noise from less impactful predictors.